Programare Python - Curs 4 2023-2024

Tipuri de date (continuare)

Dicționare (dict)

Dictionarele sunt un tip de date care poate stoca informatii conectate. Structura acestora este de perechi cheie - valoare.

Membrii dictionarului nu pastreaza nici o ordine - extragerea informatiei poate conduce la alta ordine decat adaugarea ei (pana la Python 3.7)

Dictionare: Initializare si operatii

```
md = {'one' : 12, 'two': 32, 'three': 87}
print(md)
print(type(md))

print(dir(md))

{'one': 12, 'two': 32, 'three': 87}
<class 'dict'>
['__class__', '__class_getitem__', '__contains__', '__delattr__',
'__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getitem__', '__getstate__', '__gt__',
'__hash__', '__init__', '__init_subclass__', '__ior__', '__iter__',
'__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__',
'__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__ror__',
'__setattr__', '__setitem__', '__sizeof__', '__str__',
'__subclasshook__', 'clear', 'copy', 'fromkeys', 'get', 'items',
'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
```

Initializarea unui dictionar nou:

```
masini = {}
```

Operatii elementare cu dictionarul:

```
masini['volvo'] = 1
masini['renault'] = 3


print(masini)

{'volvo': 1, 'renault': 3}
```

Intrarile din dictionare pot fi modificate:

```
masini['renault'] = 4
masini['volvo'] += 1
masini['volvo'] = masini['volvo'] + 1

print(masini)
```

```
{'volvo': 3, 'renault': 4}
```

Intrarile din dictionar pot fi inlaturate:

```
del masini['volvo']

print(masini)
```

```
{'renault': 4}
```

Ce poate fi cheie in dictionar

```
dic = {}

dic['sir'] = 1
dic[13] = 2
dic[3.14] = 3
```

tuplurile pot fi chei, cu conditia sa nu contina elemente mutabile

```
dic[(1,2)] = 4
dic[4,5] = 5

dic[(1, [4,5])] = 7
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_1471/2480939040.py in <cell line: 0>()
----> 1 dic[(1, [4,5])] = 7

TypeError: unhashable type: 'list'
```

Ca si listele, dictionarele stocheaza referinte, nu copii ale valorilor

```
lst = [1,2]
d = {"a": lst}

lst.append(3)

print(d)
```

```
{'a': [1, 2, 3]}
```

exista cazuri particulare in care se poate pierde urma referintelor - in exemplu urmator dictionarul d se modifica si in situatia in care nu exista nici o operatie evidenta asupra lui

```
d = {"a": [1,2]}

x = d["a"]
x.append(3)

print(d)

{'a': [1, 2, 3]}
```

metoda copy() a obiectului de tip dictionar produce o copie superficiala. In exemplul urmator, datorita faptului ca valoarea este o lista (referinta acesteia) metoda copy() va copia referinta iar modificarea listei se va vedea si in dictionarul original.

```
d1 = {"a": [1,2]}
d2 = d1.copy()

d2["a"].append(3)

print(d1)

{'a': [1, 2, 3]}
```

Dictionare: Traversare

Traversarea unui dictionar implica parcurgerea tuturor membrilor acestuia si efectuarea de operatii asupra lor.

```
masini['volvo'] = 3
masini['alfaromeo'] = 4
masini['lamboorghini'] = 13
print(masini)

{'renault': 4, 'volvo': 3, 'alfaromeo': 4, 'lamboorghini': 13}
```

Traversarea dictionarului poate face uz de cateva dintre metodele pe care le pune la dispozitie clasa dict:

'items', 'keys', 'values'

Acestea returneaza liste (liste de tupluri, in cazul lui items() sau liste de elemente pentru celelalte doua, care pot fi traversate cu for ... in

```
print(masini.items())
print(masini.keys())
print(masini.values())
```

```
dict_items([('renault', 4), ('volvo', 3), ('alfaromeo', 4),
('lamboorghini', 13)])
dict_keys(['renault', 'volvo', 'alfaromeo', 'lamboorghini'])
dict_values([4, 3, 4, 13])
```

```
for key, value in masini.items():
  print("Key: %s value: %s" %(key, value))
```

```
Key: renault value: 4
Key: volvo value: 3
Key: alfaromeo value: 4
Key: lamboorghini value: 13
```

key, value nu sunt cuvinte cheie obligatorii, ele se pot numi oricum:

```
for masina, cate in masini.items():
  print("masina: %s cate: %s" %(masina, cate))
```

```
masina: renault cate: 4
masina: volvo cate: 3
masina: alfaromeo cate: 4
masina: lamboorghini cate: 13
```

Traversarea se poate face si cu ajutorul metodei keys()

```
for masina in masini.keys():
  print("masina: %s cate: %s" %(masina, masini[masina]))
```

```
masina: renault cate: 4
masina: volvo cate: 3
masina: alfaromeo cate: 4
masina: lamboorghini cate: 13
```

Daca dorim sa obtinem cheile dictionarului sortate alfabetic, putem folosi functia sorted (standard in Python)

```
for masina in sorted(masini.keys()):
  print("masina: %s cate: %s" %(masina, masini[masina]))
```

```
masina: alfaromeo cate: 4
masina: lamboorghini cate: 13
masina: renault cate: 4
masina: volvo cate: 3
```

Dictionare: Sortarea dupa valoare

Sortarea dupa valoare este una dintre cele mai importante operatii care pot fi necesare in lucrul cu dictionare.

Aceasta este surprinzator de dificila - nu exista instrumente directe. Putem sa observam ca functia sorted() este ceva mai complexa decat modul in care am utilizat-o noi:

```
help(sorted)
```

```
Help on built-in function sorted in module builtins:

sorted(iterable, /, *, key=None, reverse=False)
    Return a new list containing all items from the iterable in
ascending order.

    A custom key function can be supplied to customize the sort order,
and the
    reverse flag can be set to request the result in descending order.
```

sorted(iterable, /, *, key=None, reverse=False)

/ in lista argumentelor reprezinta sfarsitul listei de parametrii pozitionali iar * reprezinta inceputul listei de parametrii denumiti, acestea doua sunt doar marcaje, nu argumente.

Daca le ignoram, obtinem:

sorted(iterable, key=None, reverse=False)

manualul spune despre parametrul key: "A custom key function can be supplied to customize the sort order".

Putem utiliza o functie anonima (lambda) pentru a modifica modul in care functioneaza procesul de sortare

```
masini_byval = sorted(masini.items(), key=lambda x: x[1])
print(masini_byval)
```

```
[('volvo', 3), ('renault', 4), ('alfaromeo', 4), ('lamboorghini', 13)]
```

se observa urmarind structura print-ului ca nu mai avem un dictionar ci o lista de tupluri. Aceasta pentru ca sort a primit de la noi masini.items() care transforma dictionarul intr-o lista de tupluri.

Pentru a obtine un dictionar trebuie sa folosim functia dict

```
masini_sortate_value = dict(masini_byval)
print (masini_sortate_value)
```

```
{'volvo': 3, 'renault': 4, 'alfaromeo': 4, 'lamboorghini': 13}
```

Dictionare: comprehension

Ca si in cazul listelor, dictionary comprehension reprezinta o serie de tehnici care tin de sintaxa limbajului de a obtine in foarte putine linii de cod ceea ce ar putea fi obtinut in blocuri semnificativ mai mari

Dictionary comprehension implica initializarea dictionarelor folosind blocuri de cod in loc de perechi cheie - valoare

structura generala este astfel:

in loc sa scriem dictionar_nou = {'cheie': 'valoare'} inlocuim una dintre cele doua (sau pe ambele) cu un bloc de cod.

dictionar_nou = {cheie: for ... if ...}

De exemplu, daca dorim sa inversam cheile cu valorile:

```
swapped_dict = {value: key for key, value in masini.items()}

print(swapped_dict)

{4: 'alfaromeo', 3: 'volvo', 13: 'lamboorghini'}
```

Adaugarea unei unitati la toate valorile unui dictionar:

```
mai_multe_masini = {masina: numar + 1 for masina, numar in
masini.items()}
print(mai_multe_masini)

{'renault': 5, 'volvo': 4, 'alfaromeo': 5, 'lamboorghini': 14}
```

evident, oricand se poate recurge la metoda clasica:

```
mai_multe_masini = {}

for masina, numar in masini.items():
  mai_multe_masini[masina] = numar + 1

print(mai_multe_masini)

{'renault': 5, 'volvo': 4, 'alfaromeo': 5, 'lamboorghini': 14}
```

expresiile pot fi mai complicate - ele pot contine conditionale De exemplu, sa presupunem ca vrem sa mai adaugam 1 la valorile din mai_multe_masini dar doar pentru cele care au valoarea mai mica de 10.

Varianta clasica arata astfel:

```
mm_masini = {}

for masina, numar in mai_multe_masini.items():
  if numar < 10:
    mm_masini[masina] = numar + 1
  else:
    mm_masini[masina] = numar
```

```
print(mm_masini)
```

```
{'renault': 6, 'volvo': 5, 'alfaromeo': 6, 'lamboorghini': 14}
```

varianta in care folosim dictionary comprehension:

```
mm_masini_dc = {masina: numar + 1 if numar < 10 else numar for masina,
numar in mai_multe_masini.items()}
print (mm_masini_dc)
```

```
{'renault': 6, 'volvo': 5, 'alfaromeo': 6, 'lamboorghini': 14}
```

un alt exemplu: conversia de la farenheit la celsius:

vom itera peste tot dictionarul farenheit si vom folosi la fiecare item formula de conversie:

(5/9) * (F-32)

cheile raman nemodificate, valorile se modifica conform formulei

```
fahrenheit = {'t1': -30,'t2': -20,'t3': -10,'t4': 0}

celsius = {k:(float(5)/9)*(v-32) for (k,v) in fahrenheit.items()}

print(celsius)
```

```
{'t1': -34.44444444444444, 't2': -28.88888888888889, 't3': -
23.333333333333336, 't4': -17.77777777777778}
```

Putem adauga oricate conditii la dictionary comprehension

```
print(mai_multe_masini)

mm_masini_dc_2 = {masina: numar for masina, numar in
mai_multe_masini.items() if numar >4  if masina.startswith('l') }
print(mm_masini_dc_2)
```

```
{'renault': 5, 'volvo': 4, 'alfaromeo': 5, 'lamboorghini': 14}
{'lamboorghini': 14}
```

observam ca aparitia lui else modifica sintaxa impingand for-ul pe ultimul loc. Daca dorim sa folosim mai multe if-uri for-ul trebuie sa vina inaintea acestora si nu mai putem folosi else

Frozen sets

Un set inghetat (frozen set) este un tip special de set care nu mai poate fi modificat dupa ce a fost creat.

```python
my_frozen_set = frozenset([1, 2, 3, 4])

dir(my_frozen_set)
```

```
['__and__',
 '__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__ne__',
 '__new__',
 '__or__',
 '__rand__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__ror__',
 '__rsub__',
 '__rxor__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__sub__',
 '__subclasshook__',
 '__xor__',
 'copy',
 'difference',
 'intersection',
 'isdisjoint',
 'issubset',
 'issuperset',
 'symmetric_difference',
 'union']
```

Se observa ca setul inghetat are majoritatea proprietatilor unui set normal, cu exceptia celor care l-ar putea modifica (add(), remove(), pop())

La ce sunt utile? Una dintre utilizari sunt dictionarele cu cheie multipla in care ordinea nu conteaza.

```
my_dict = {
    frozenset({'Ion', 1984, 'Bacau'}): "data1",
    frozenset({'Vasile', 1987, 'Vaslui'}): "data2",
    frozenset({'Monica', 1994, 'Sibiu'}): "data3"
}

print(my_dict)
```

```
{frozenset({1984, 'Ion', 'Bacau'}): 'data1', frozenset({1987,
'Vaslui', 'Vasile'}): 'data2', frozenset({'Monica', 1994, 'Sibiu'}):
'data3'}
```

Cache al rezultatelor unei functii

```
import time

cache = {}

def compute_cost(page, category, price_min, price_max):
    time.sleep(2)
    return page * 10 + (price_max - price_min) #expresie oarecare


calls = [
    {"page": 1, "category": "books",       "price_min": 10,
"price_max": 100},
    {"category": "electronics", "price_max": 300, "price_min": 50,
"page": 2},
    {"price_max": 100, "page": 1, "category": "books", "price_min":
10},   # repeat (different order)
    {"price_min": 5, "category": "toys", "page": 3, "price_max": 80},
    {"category": "clothes", "page": 1, "price_max": 120, "price_min":
20},
    {"price_min": 50, "page": 2, "price_max": 300, "category":
"electronics"},   # repeat (different order)
    {"page": 4, "price_max": 90, "category": "books", "price_min":
15},
    {"category": "garden", "price_min": 30, "price_max": 200, "page":
5},
    {"category": "toys", "price_max": 80, "price_min": 5, "page": 3},
# repeat (different order)
    {"price_min": 25, "category": "sports", "page": 1, "price_max":
150},
]
```

```
for i, params in enumerate(calls, start=1):
    key = frozenset(params.items())

    start = time.time()

    if key in cache:
        result = cache[key]
        source = "cache"
    else:
        result = compute_cost(**params)
        cache[key] = result
        source = "computed"

    elapsed = time.time() - start

    print(f"call {i}: {source:8} -> result={result},
time={elapsed:.3f}s")
```

```
call 1: computed -> result=100, time=2.000s
call 2: computed -> result=270, time=2.000s
call 3: cache    -> result=100, time=0.000s
call 4: computed -> result=105, time=2.000s
call 5: computed -> result=110, time=2.000s
call 6: cache    -> result=270, time=0.000s
call 7: computed -> result=115, time=2.000s
call 8: computed -> result=220, time=2.000s
call 9: cache    -> result=105, time=0.000s
call 10: computed -> result=135, time=2.000s
```

Control flow

Range

range este un obiect care descrie o regulă de generare a unei secvențe de numere întregi, definită prin start, stop și pas, fără a stoca efectiv numerele

```
rng = range(1,20)
print(type(rng))

print (dir(rng))

listfr = [*rng]

print (listfr)
print (type(listfr))
```

```
<class 'range'>
['__bool__', '__class__', '__contains__', '__delattr__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
```

```
'__getitem__', '__getstate__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__iter__', '__le__', '__len__', '__lt__',
'__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
'__reversed__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__', 'count', 'index', 'start', 'step', 'stop']
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
<class 'list'>
```

range nu genereaza propriu-zis o lista ci un obiect din clasa range, cu propriile metode. range poate fi convertit la lista cu ajutorul operatorului unpacking *

Finalul intervalului determinat de range nu este atins

```
for i in range(1,10):
  print(i)

1
2
3
4
5
6
7
8
9
```

Daca nu specificam inceputul, acesta este 0

```
for i in range(10):
  print(i)

0
1
2
3
4
5
6
7
8
9
```

putem specifica pasul. Daca o facem, trebuie sa trecem si inceputul, altfel Python nu stie daca nu am trimis start - stop

```
for i in range(1, 10,2):
  print(i)
```

```
1
3
5
7
9
```

pasul negativ implica traversarea intervalului in sens invers

```
for i in range(10, -1,-1):
  print(i)

10
9
8
7
6
5
4
3
2
1
0
```

In python 2, range genera toate elementele la rulare. Acest comportament determina utilizari semnificative ale memoriei iar daca range era utilizat doar pentru a itera printr-un set de date existent, acea memorie era consumata degeaba. In Python2 exista functia xrange care genera elementele unul cate unul. In python 3, range se comporta ca xrange din python 2 asa ca xrange nu mai este necesar.

Prin comparare cu generarea unei liste complete de numere intregi, range este mult mai rapid

```
import time
import tracemalloc

N = 10_000_000
INDEX = 9_000_000

# -------- LIST --------
tracemalloc.start()
t0 = time.time()

lst = list(range(N))              # construieste intreaga lista
value_list = lst[INDEX]           # acceseaza un index mare

t1 = time.time()
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

print("LIST")
```

```
print("value:", value_list)
print("time:", round(t1 - t0, 3), "s")
print("memory (MB):", round(peak / 1024 / 1024, 2))


# -------- RANGE --------
tracemalloc.start()
t0 = time.time()

r = range(N)                        # nu se creaza lista
value_range = r[INDEX]              # accesarea unui index mare din range

t1 = time.time()
current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

print("\nRANGE")
print("value:", value_range)
print("time:", round(t1 - t0, 3), "s")
print("memory (MB):", round(peak / 1024 / 1024, 2))
```

```
LIST
value: 9000000
time: 11.258 s
memory (MB): 381.48

RANGE
value: 9000000
time: 0.0 s
memory (MB): 0.02
```

if - elif - else

Sintaxa generala a if-urilor este

if expresie: bloc de cod

```
x = 1
if x > 0:
  print("x este pozitiv")
```

```
x este pozitiv
```

O particularitate a limbajului Python este reprezentata de operatorii **is** si **is not** care testeaza referintele obiectelor

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
```

```
print(id(a))
print(id(b))
print(id(c))

print(a is b)  # Fals - a si b au referinte catre obiecte diferite
print(a is c)  # Adevarat - a si c au referinte catre acelasi obiect
```

```
132846026824064
132846082887232
132846026824064
False
True
```

Se observa ca la crearea unei noi liste, Python va crea un nou obiect de fiecare data, indiferent daca lista are aceeasi membri. Atribuirea pastreaza referinta pentru ca listele vor ramane aceleasi (daca se adauga un element uneia dintre ele se va adauga si celeilalte). Pentru copierea unei liste in alta variabila se va folosi list() sau sliciing

```
da = [1,2,3]

fa = da

print(da)
print(fa)

fa.append(4)

print(da)
print(fa)
```

```
[1, 2, 3]
[1, 2, 3]
[1, 2, 3, 4]
[1, 2, 3, 4]
```

Alte operatoare importante, existente si in alte limbaje de programare sunt **in** si **not in** care testeaza prezenta unui element intr-o lista

```
animale = ['pisica', 'caine', 'elefant','ornitorinc']

print('pisica' in animale)    #True
print('pisica' not in animale)  #False
print('gaina' in animale)  #True
print('gaina' not in animale)  #False
```

```
True
False
False
True
```

```
x = -5
if x == 0:
    print(x,"este zero")
elif x > 0:
    print(x,"este positiv")
else:
    print(x,"este negativ")


print('Final if')
```

```
-5 este negativ
Final if
```

Conditiile sunt evaluate de sus in jos iar executia se opreste la prima conditie adevarata. In exemplul urmator, cea de-a doua conditie, desi adevarata, nu ruleaza

```
x = 10

if x > 0:
    print("A")
elif x > 5:
    print("B")
```

```
A
```

daca dorim sa rulam altele comparatii trebuie sa folosim doua if-uri unul dupa celalalt

```
if x > 0:
    print("A")

if x > 5:
    print("B")
```

```
A
B
```

Multe bucle if se bazeaza pe evaluarea valorii de adevar/fals a unei expresii. In general:

- False: 0, containere goale, None
- True: orice altceva

```
print(bool(0))        # False
print(bool(0.0))      # False
print(bool(0j))       # False

print(bool(""))       # False
print(bool([]))       # False
print(bool({}))       # False
print(bool(set()))    # False
```

```
print(bool(None))     # False
```

```
False
False
False
False
False
False
False
False
```

O lista care contine un numar intreg 0 nu este o lista goala - va genera True. O lista care contine un sir de caractere gol nu este o lista goala - va genera True

```
print(bool([0])) # True
```

```
True
```

```
print(bool([""])) # True
```

```
True
```

Evaluarea unei conditii booleene se opreste la prima conditie adevarata. In codul urmator, functia f fiind cea de-a doua parte a conditiei, nu este niciodata evaluata - conditia se opreste la X care este False iar False AND orice = False

```
def f():
    print("f called")
    return True

x = 0

if x and f():
    print("if")
else:
    print("else")
```

```
else
```

Inlocuind conditia cu OR determinam evaluarea ambelor conditii

```
if x or f():
    print("if")
else:
    print("else")
```

```
f called
if
```

Python ofera o sintaxa similara operatorului ternar din c

```
condition = True
x = 10 if condition else 20
print(x)

10
```

bucle for

Sintaxa generala a buclelor for in Python este:

```
for <var> in <iterable>:
    bloc de cod
```

poate fi una sau mai multe, in functie de ceea ce returneaza iterabilul este orice obiect care are metoda **__iter__** sau orice obiect care permite accesul secvential si contine metoda **__getitem__**

Cele mai frecvente iterabile in Python sunt listele, tuplurile, seturile, dictuonarele si sirurile de caractere. De asemenea, Python ofera o serie de functii care genereaza secvente de numere (sau chiar iteratoare), cum ar fi range, xrange.

Modulul **itertools** este special conceput pentru crearea de iteratoare cat mai eficiente posibil.

bucla for in Python nu seamana deloc cu bucla for in c. Ar fi fost mai corect sa fie numita **foreach** pentru ca este de fiecare data o parcurgere iterativa a unei colectii existente sau accesarea secventiala a unui iterabil.

```
total = 0
for item in [1, 3, 5]:
    total = total + item

print(total)

9
```

O alta functie utila este "enumerate" - functie care retuirneaza un iterator din care se pot extrage atat indexul cat si valoarea elementelor

Iteratorul generat de enumerate returneaza un tuplu cu pozitia si valoarea. Forul poate contine o singura variabila, caz in care aceasta va lua valoarea unui tuplu sau doua variabile, caz in care tuplul va fi automat despachetat.

```
lista = [2,3,4,5,6,7]

for poz, val in enumerate(lista):
  print(poz,val)

0 2
1 3
```

```
2 4
3 5
4 6
5 7
```

```
lista = [2,3,4]

for var in enumerate(lista):
  print(var)
  print(type(var))
```

```
(0, 2)
<class 'tuple'>
(1, 3)
<class 'tuple'>
(2, 4)
<class 'tuple'>
```

Pentru a obtine comportamentul asteptat al buclei for din alte limbaje de programare (cel in care puteam decide un interval al unei variabile de ciclare si un anumit pas al acestuia) folosim functia range

```
for i in range(8):
    print(i)
```

```
0
1
2
3
4
5
6
7
```

```
for i in range(8,16):
    print(i)
```

```
8
9
10
11
12
13
14
15
```

```
for i in range(8,0,-1):
    print(i)
```

```
8
7
```

```
6
5
4
3
2
1
```

variabila i NU controleaza iteratia, ca in C, php, etc. Reatribuirea unai alte valori in interiorul buclei nu schimba programul.

```
for i in range(5):
    print("before:", i)
    i = 100
    print("after :", i)
```

```
before: 0
after : 100
before: 1
after : 100
before: 2
after : 100
before: 3
after : 100
before: 4
after : 100
```

Range + formule

```
squares = [d**2 for d in range(1,11)]
print(squares)
```

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

break + continue

Break intrerupe executia forului imediat superior

```
for i in range(5):
    if i == 3:
        break
    print(i)

print("post break")
```

```
0
1
2
post break
```

Instructiunea break poate fi pozitionata in oricate secvente if - ea va intrerupe bucla imediat superioara

```
for i in range(5):
    if i == 3:
      if i > 0:
        if i < 100:    #break apare aici - toate celelalte conditii
sunt adevarate
          if i < 1000:
            if i < 10000:
              if i ** 2 < 200:
                break
    print(i)

print("post break")
```

```
0
1
2
post break
```

in codul urmator, break nu intrerupe bucla exterioara (cu j) ci doar pe cea interioara. j ajunge pana la 4 (cat ii permite range-ul) dar i nu depaseste niciodata 2

```
for j in range(5):
  for i in range(5):
      if i == 3:
        print('breaking')
        break
      print(j, i)

print("post if")
```

```
0 0
0 1
0 2
breaking
1 0
1 1
1 2
breaking
2 0
2 1
2 2
breaking
3 0
3 1
3 2
breaking
4 0
```

```
4 1
4 2
breaking
post if
```

Continue

Continue "sare" peste un ciclu al iteratiei la urmatorul. In urmatorul exemplu, cand i devine eval cu 3, instructiunea "continue" sare peste print(i) si determina solicitarea urmatoarei valori a lui i (4) de la iterator.

```
for i in range(5):
    if i == 3:
        continue
    print(i)

0
1
2
4
```

Un exemplu destul de tipic al utilizarii continue in python este ciclarea printr-o lista mixta, in care doar unii din membrii se preteaza anumitor prelucrari

```
data = [1, 2, None, 4, "error", 6, None, 8]

for item in data:
    # Daca elementul curent este un sir de caractere sau None, sarim
la urmatoarea iteratie
    if item is None or isinstance(item, str):
        continue
    # altfel, inmultim cu 2
    else:
        print(item * 2)

2
4
8
12
16
```

for ... else??

Python permite adaugarea unui else la o bucla for. Aceasta se va executa la terminarea iteratiei, cu exceptia cazului in care s-a folosit conditia break.

```
for i in range(4):
  print(i)
else:
```

```
  print('stop')

print('post loop')
```

```
0
1
2
3
stop
post loop
```

O utilizare frecventa a acestui else este in cazurile in care avem o serie de numere pe care dorim sa le testam iar testul implica o bucla for interna. Un exemplu frecvent este determinarea unui set de numere prime, intr-o lista de numere date sau un anumit range.

Bucla for exterioara (n) trece prin toate numerele care vor fi testate

Bucla interioara (care face parte din test) trebuie sa verifice daca numarul curent se divide exact cu oricare dintre numerele mai mici decat el. Daca aceasta bucla interna se termina fara break, putem folosi un else pentru a anunta numarul prim.

```
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, 'equals', x, '*', n//x)
            break
    else:
        # loop fell through without finding a factor
        print(n, 'is a prime number')
```

```
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

O alta utilizare este in cautari a unor elemente in liste sau altfel de colectii. In codul urmator else se va executa doar daca numarul cautat nu a fost gasit

```
numere = [1, 3, 5, 7]
search_for = 4

for num in numere:
    if num == search_for:
        print(f"Found {search_for}!")
        break
```

```
else:
    print(f"{search_for} not found in the list.")
```

```
4 not found in the list.
```

for .. else a fost propus si pentru ECMA script in 2018 dar a fost respins

zip

functie care preia mai multe iterabile si construieste tupluri din ele. Denumirea sugereaza un fermoar.

```
a = [1, 2, 3]
b = ["a", "b", "c"]

for x, y in zip(a, b):
    print(x, y)
```

```
1 a
2 b
3 c
```

Daca cele doua iterabile contin elemente diferite, zip se opreste la ultima pereche care poate fi constituita.

```
a = [1, 2, 3, 4, 5]
b = ["a", "b", "c"]

print(list(zip(a, b)))
```

```
[(1, 'a'), (2, 'b'), (3, 'c')]
```

zip poate compute mai multe iterabile

```
names = ["Ana", "Dan", "Mihai"]
scores = [10, 8, 9]
cities = ["București", "Cluj", "Iași"]

for name, score, city in zip(names, scores, cities):
    print(name, score, city)
```

```
Ana 10 București
Dan 8 Cluj
Mihai 9 Iași
```

while

Ca in alte limbaje de programare, while este o buicla utilizata cand nu se cunoaste numarul de iteratii dinainte.

```
while expresie:
  bloc de cod

suma = 0
i = 1
while i <= 10:
    suma += i
    i += 1

print(suma)

55
```

break, continue

```
i=0
while i <= 10:
    print (i)
    i += 1
    if i > 5:
       break

0
1
2
3
4
5

i = 0
while i < 10:
  i+= 1
  if i == 3:
    continue
  print(i)

1
2
4
5
6
7
8
9
10
```

continue poate fi periculos in combinatie cu while. In codul urmator, faptul ca incrementarea variabile i se intampla dupa continue face ca incrementarea sa fie ocolita iar bucla ruleaza la infinit.

```
i = 0
while i < 10:
    if i == 2:
        continue
    i += 1
    print(i)
```

```
1
2

---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_1471/830411151.py in <cell line: 0>()
      2 while i < 10:
      3     if i == 2:
----> 4         continue
      5     i += 1
      6     print(i)

KeyboardInterrupt:
```

while ... else ?

```
contor = 0
while contor < 3:
    contor += 1
    print('in interiorul buclei')
else:
    print('in exteriorul buclei')
```

```
in interiorul buclei
in interiorul buclei
in interiorul buclei
in exteriorul buclei
```

else nu ruleaza decat daca bucla se termina normal, fara break

```
contor = 0
while contor < 3:
    contor += 1
    print("in bucla")
else:
    print("fara break")
```

```
in bucla
in bucla
in bucla
fara break
```

```
contor = 0
while contor < 3:
    contor += 1
    print("in bucla")
    if contor == 2:
        break
else:
    print("fara break")
```

```
in bucla
in bucla
```

match

Match este echivalentul Python al declaratiei "switch" din alte limbaje de programare

```
status = 400

match status:
  case 200:
    print("all ok")
  case 400:
    print ("Bad request")
  case 404:
    print("Not found")
  case 418:
    print("I'm a teapot")
  case _:
    print("Something's wrong with the internet")
```

```
Bad request
```

Underscore (_) este folosit pentru a capta situatiile in care nu exista un match anterior. Acesta este numit "irrefutable pattern" si trebuie pozitionat pe ultimul loc, altfel capatam eroarea:

SyntaxError: wildcard makes remaining patterns unreachable

```
status = 400

match status:
  case _:
    print("Something's wrong with the internet")
  case 200:
    print("all ok")
  case 400:
    print ("Bad request")
  case 404:
    print("Not found")
```

```
  case 418:
    print("I'm a teapot")
```

```
  File "/tmp/ipykernel_1471/4281611498.py", line 4
    case _:
         ^
SyntaxError: wildcard makes remaining patterns unreachable
```

match poate fi folosit si pentru compararea structurii

```
data = (10, 20)

match data:
    case (x, y):
        print(x, y)
```

```
10 20
```

```
point = (0, 5)

match point:
    case (0, y):
        print("on Y axis:", y)
```

```
on Y axis: 5
```

```
data = [1, 2, 3, 4]

match data:
    case [first, *rest]:
        print(first, rest)
```

```
1 [2, 3, 4]
```

```
user = {"name": "Ana", "age": 25}

match user:
    case {"name": name}:
        print(name)
```

```
Ana
```

conditiile pot include si comparatii

```
items = [
    (10, 20),
    (7, 3),
    (5, 5),
    (1, 9),
    (100,),
```

```
    "abc",
    (8, 2),
]

for data in items:
    match data:
        case (x, y) if x < y:
            print(data, "-> increasing")
        case (x, y) if x > y:
            print(data, "-> decreasing")
        case (x, y):
            print(data, "-> equal")
        case _:
            print(data, "-> not a pair")
```

```
(10, 20) -> increasing
(7, 3) -> decreasing
(5, 5) -> equal
(1, 9) -> increasing
(100,) -> not a pair
abc -> not a pair
(8, 2) -> decreasing
```

all and any

all si any sunt functii care evalueaza toate elementele dintr-o lista in cautarea unei valori de adevar.

- all returneaza true daca toate elementele listei sunt adevarate
- any returneaza true daca unul dingur dintre elementele listei sunt adevarate.

ambele functii pot fi utilizate intr-o maniera similara cu expresiile de comprehension

```
numere = [1,2,3,4,5,6,7,8,9]
pare = [2,4,6,8]


all_numere_pare = all(num % 2 == 0 for num in numere)
any_numere_pare = any(num % 2 == 0 for num in numere)

all_pare_pare = all(num % 2 == 0 for num in pare)
any_pare_pare = any(num % 2 == 0 for num in pare)

print (all_numere_pare)   # NU toate numerele sunt pare
print (any_numere_pare)  # cel putin un numar e par

print (all_pare_pare)   # toate numerele sunt pare
print (any_pare_pare)   # exista cel putin un numar par
```

```
False
True
True
True
```

pass

pass este o instructiune care nu face nimic, fara a genera erori. Este un inlocuitor sintactic pentru cazul in care scriem functii sau bucle care raman pentru moment goale.

Urmatorul cod va genera o eroare:

`IndentationError: expected an indented block after 'for' statement on line 1`

```
for nr in range(10):


for h in range(5):
  print(h)

  File "/tmp/ipykernel_1471/3743435674.py", line 4
    for h in range(5):
    ^
IndentationError: expected an indented block after 'for' statement on line 1
```

adaugarea instructiunii pass va permite executia acestuia

```
for nr in range(10):
  pass

for h in range(5):
  print(h)

0
1
2
3
4
```

try .. except

try .. except este o contructie de tip flow-control care este utilizata pentru a "capta" erorile. Sunt multe fragmente de cod care pot genera mai multe feluri de erori, uneori din surse neasteptate.

except poate alege eroarea pe care sa o capteze. Este posibila insirarea mai multor clause except cu mai multe erori pentru a trata diferentiat erorile.

metoda except poate prelua obiectul eroare care are de multe ori un mesaj disponibil.

Cateva cazuri in care se utilizeaza verificarea erorilor sunt

Operatii I/O

```
try:
  with open("myfile.txt", "r") as file:
    lines = file.readlines()

except FileNotFoundError as e:
  print("Nu am gasit fisierul")
  print(type(e))   # fiecare eroare este un obiect
  print(str(e))   #conversia in str a obiectului eroare conduce de
obicei catre un mesaj explicit

except IOError:
  print("Eroare in timpul citirii fisierului")
```

```
Nu am gasit fisierul
<class 'FileNotFoundError'>
[Errno 2] No such file or directory: 'myfile.txt'
```

Pericol de impartire la zero

```
nr = 0; #variabila care vine dintr-o sursa externa

try:
  rezultat = 10/nr

except ZeroDivisionError as e:
  print("Impartire la zero nu se poate")
  print(str(e))
```

```
Impartire la zero nu se poate
division by zero
```

Acces la API-uri externe

```
import requests

try:
  response = requests.get("https://api.chucknorris.io/jokes/random")
  print(response.text)
except requests.RequestException:
  print("Solicitare API nereusita")
```


```
{"categories":[],"created_at":"2020-01-05
13:42:30.177068","icon_url":"https://api.chucknorris.io/img/avatar/
chuck-norris.png","id":"ZxRSFBR_STiXV4ZGSMhBSA","updated_at":"2020-01-
05
13:42:30.177068","url":"https://api.chucknorris.io/jokes/ZxRSFBR_STiXV
```

```
4ZGSMhBSA","value":"Chuck Norris can bring a bike to a monster truck show"}
```

Se pot utiliza si clauze except fara tipul erorii. Acestea vor capta toate erorile

```
try:
  with open("myfile.txt", "r") as file:
    lines = file.readlines()

except:
  print("Nu am gasit fisierul")
```

```
Nu am gasit fisierul
```

```
try:
  with open("myfile.txt", "r") as file:
    lines = file.readlines()

except Exception as e:
  print("Nu am gasit fisierul")
  print(type(e))   # fiecare eroare este un obiect
  print(str(e))   #conversia in str a obiectului eroare conduce de obicei catre un mesaj explicit
```

```
Nu am gasit fisierul
<class 'FileNotFoundError'>
[Errno 2] No such file or directory: 'myfile.txt'
```

finally

Finally poate fi adaugat ca parte a combinatiei try .. except ca un bloc de cod care se va executa indiferent de consecinta blocului try (eroare sau nu). Blocul finally este optional. Este util in cazul in care trebuie facuta "curatenie" - ex. inchiderea conexiunilor catre o baza de date.

```
try:
  with open("myfile.txt", "r") as file:
    lines = file.readlines()

except Exception as e:
  print(str(e))   #conversia in str a obiectului eroare conduce de obicei catre un mesaj explicit

finally:
  print("Indiferent ce s-a intamplat, s-a intamplat")
```

```
[Errno 2] No such file or directory: 'myfile.txt'
Indiferent ce s-a intamplat, s-a intamplat
```

try ... except ... else ?

else se poate adauga dupa if .. except. Blocul de cod din else se va executa doar daca nu exista nici o eroare (prin opozitie cu finally care se va executa oricum).

```
nr = 0; #variabila care vine dintr-o sursa externa

try:
  rezultat = 10/nr

except ZeroDivisionError as e:
  print(str(e))

else:
  print("dupa except")

division by zero
```

Functii (subrutine, subprograme)

Functiile sunt elemente fundamentale ale programarii procedurale care permit impartirea codului in fragmente reutilizabile si implementarea unor algoritmi care nu ar fi posibili altfel (recursivi).

Definirea unei functii in Python trebuie sa tina seama de urmatoarele:

```
- Funtia trebuie sa fie definita inainte de apelarea ei
- denumirea functiei trebuie sa fie precedata de cuvantul cheie "def"
- dupa denumirea functiei, in paranteze rotunde si separate prin
virgula se trec argumentele acesteia
- la finalul liniei de definitie trebuie pus doua puncte
- Intreg corpul functiei (blocul de cod) trebuie indentat fata de
definirea acesteia si fata de ceea ce urmeaza

def palindrome(s):
    return s.replace(" ","").lower() == s[::-1].replace("
","").lower()

print(palindrome("Borrow or rob"))

True
```

Functia ca obiect

Functia este un obiect in Python, obiectele au metode, cum am vazut, metodele se obtin cu dir()

```
print(dir(palindrome))

['__annotations__', '__builtins__', '__call__', '__class__',
'__closure__', '__code__', '__defaults__', '__delattr__', '__dict__',
```

```
'__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__get__',
'__getattribute__', '__getstate__', '__globals__', '__gt__',
'__hash__', '__init__', '__init_subclass__', '__kwdefaults__',
'__le__', '__lt__', '__module__', '__name__', '__ne__', '__new__',
'__qualname__', '__reduce__', '__reduce_ex__', '__repr__',
'__setattr__', '__sizeof__', '__str__', '__subclasshook__',
'__type_params__']
```

`__annotations__`

returneaza sub forma unui dictionar o serie de informatii suplimentare pe care le pot capata parametrii functiilor, de pilda type hints

```
def func_th(x: int, y: str) -> float:
    return x * len(y) / 2.0

print(func_th.__annotations__)

{'x': <class 'int'>, 'y': <class 'str'>, 'return': <class 'float'>}
```

`__call__`

Call ofera un mod alternativ de a apela functia

```
print(func_th(3,"scoica"))
print(func_th.__call__(3,"scoica"))

9.0
9.0
```

`__class__`

Ca si la celelalte obiecte, returneaza clasa care a instantiat obiectul

```
print(func_th.__class__)

<class 'function'>
```

`__code__` permite accesul la bytecodul compilat al functiei respective, Acesta are o serie de atribute, dintre care:

co_argcount Numar intgreg care reprezinta numarul de argumente pozitionale fize pe care functia le asteapta

co_code: Un tuplu care contine bytecodul corpului functiei

co_consts: Un tuplu care contine constantele

co_filename: un string care contine numele fisierului in care functia a fost definita

co_firstlineno: un numar intreg care reprezinta linia la care incepe functia in fisierul obtinut cu atributul anterior, co_filename

co_varnames: un tuplu de siruri care arata numele argumentelor functiei

co_name: sir care afiseaza numele functiei

co_names: tuplu care contine sub fgorma de siruri toate numele variabilelor globale si nonlocale disponibile pentru referinte in corpul functiei

```
print(func_th.__code__)

<code object func_th at 0x78d2a030f3c0, file
"/tmp/ipykernel_1471/1602289743.py", line 1>

print(func_th.__code__.co_code)

b'\x97\x00|\x00t\x01\x00\x00\x00\x00\x00\x00\x00\x00|\x01\xab\x01\x00\
x00\x00\x00\x00\x00z\x05\x00\x00d\x01z\x0b\x00\x00S\x00'

print(func_th.__code__.co_names)

('len',)
```

Functiile care fac parte din biblioteca de baza nu contin metoda `__code__` - acestea nu sunt compilate in bytecode.

```
print(dir(print))

['__call__', '__class__', '__delattr__', '__dir__', '__doc__',
'__eq__', '__format__', '__ge__', '__getattribute__', '__getstate__',
'__gt__', '__hash__', '__init__', '__init_subclass__', '__le__',
'__lt__', '__module__', '__name__', '__ne__', '__new__',
'__qualname__', '__reduce__', '__reduce_ex__', '__repr__', '__self__',
'__setattr__', '__sizeof__', '__str__', '__subclasshook__',
'__text_signature__']
```

Dar functiile importate din module instalate ulterior (ex: cu ajutporul pip) pot fi inspectate

```
import requests

print(requests.get.__code__.co_filename)
print(requests.get.__code__.co_firstlineno)

/usr/local/lib/python3.12/dist-packages/requests/api.py
62
```

Python permite dezasamblarea bytecodului pentru a putea observa operatiile masinii virtuale. Pentru aceasta, trebuie importat modului **dis**

```
import dis

def msm(a,b):
  return a+b


print(msm.__code__.co_code)
print('-----')
print(dis.dis(msm))
```

```
b'\x97\x00|\x00|\x01z\x00\x00\x00S\x00'
-----
  3           0 RESUME                   0

  4           2 LOAD_FAST                0 (a)
              4 LOAD_FAST                1 (b)
              6 BINARY_OP                0 (+)
             10 RETURN_VALUE
None
```

Sunt patru operatii in codul dezasamblat. Masina virtuala Python este o masina bazata pe o stiva, primele doua operatii trimit cele doua argumente in stiva, cea de-a treia va adauga cele doua valori si va trimite automat rezultatul in stiva iar return_value va returna valoarea de pe pozitia cea mai inalta in stiva.

Se observa diferente intre acest "assembler" si cex x86 pentru ca assemblerul x86 este hibrid, bazat atat pe o stiva cat si pe registre iar CPython este bazat doar pe stiva.

Modul de transmitere al argumentelor

Noțiunile de argument și parametru sunt adesea folosite interschimbabil, dar există o diferență:

- parametrii apar în definiția funcției
- argumentele sunt valorile transmise la apelul funcției

În multe limbaje de programare se discută despre două moduri de transmitere a argumentelor:

- by value (prin valoare)
- by reference (prin referință)

În Python, această distincție nu se aplică în mod direct.

În Python, toate variabilele sunt nume care fac referire la obiecte din memorie

La apelul unei funcții:

- parametrii devin nume locale care se leagă la aceleași obiecte ca argumentele
- Nu se creează automat copii ale obiectelor.

Anumite tipuri de date sunt imutabile:

- numere (int, float, etc.)

- șiruri de caractere
- tupluri
- frozenset

Altele sunt mutabile:

- liste
- dicționare
- seturi

Diferența de comportament apare din mutabilitatea obiectelor, nu din modul de transmitere:

- obiectele imutabile nu pot fi modificate → orice "modificare" creează un obiect nou
- obiectele mutabile pot fi modificate → modificările sunt vizibile și în exterior

De aceea:

- pentru tipurile imutabile, comportamentul pare similar cu by value
- pentru tipurile mutabile, comportamentul pare similar cu by reference

Daca trimitem un numar intreg catre o functie dar nu il modificam in interiorul acesteia, referintele raman aceleasi.

```
def changeme1(vr):
  print(vr, id(vr))
  vr = 15
  print(vr, id(vr))

a = 30
print(a, id(a))
changeme1(a)
print(a, id(a))
```

```
30 11646280
30 11646280
15 11645800
30 11646280
```

daca trimitem catre functie o lista care contine un singur numar intreg si il modificam in lista, modificarile se vor mentine dupa iesirea din functie, pentru ca lista este un tip de date mutabil.

```
def changeme2(vrl):
  print(vrl, id(vrl))
  vrl[0] = 15
  print(vrl, id(vrl))

a = [30]
print(a, id(a))
changeme2(a)
print(a, id(a))
```

```
[30] 132845688612416
[30] 132845688612416
[15] 132845688612416
[15] 132845688612416
```

```
def changeme3(vrl2):
  print(vrl2, id(vrl2))
  vrl2.append(15)
  print(vrl2, id(vrl2))

a = [30]
print(a, id(a))
changeme3(a)
print(a, id(a))
```

```
[30] 132846025622592
[30] 132846025622592
[30, 15] 132846025622592
[30, 15] 132846025622592
```

Exista vreo posibilitate de a trimite o lista astfel incat sa nu fie afectata de modificarile in interiorul functiei, emuland tipul de trimitere "by value"? Pentru a obtine acest efect trebuie sa facem undeva o copie a listei, aceasta trebuie sa fie completa (deep copy) daca e nevoie iar copia trebuie facuta explicit din cod.

Toate operatiile native din Python de copiere a listelor sunt shalow copy. Acestea se pot folosi pentru trimiterea unei liste by_value dar trebuie sa nu uitam ca acea copiere este shallow_copy

- slicing: changeme3(a[:])
- list(): changeme3(list(a))
- list.copy(): changeme3(a.copy())

toate cele trei variante au acelasi efect:

```
a = [30]
print(a, id(a))
changeme2(a.copy()) # nu modificam lista primita ca argument ci o
copie
print(a, id(a))
```

```
[30] 132846025297600
[30] 132846029674368
[15] 132846029674368
[30] 132846025297600
```

dar daca lista contine o referinta (de exemplu catre o alta lista) si modificarea afecteaza tocmai acea referinta, modificarea va ramane dupa terminarea executiei functiei.

```
def changeme4(vr):
  print(vr, id(vr))
```

```
  vr[1][0] = 4
  print(vr, id(vr))


a = [30, [15]]
print(a, id(a))
changeme4(a.copy())
print(a, id(a))
```

```
[30, [15]] 132845688612416
[30, [15]] 132846026969920
[30, [4]] 132846026969920
[30, [4]] 132845688612416
```

Exista posibilitatea de a incurca borcanele semnificativ de rau, trimitand o lista cu referinte sub forma de shallow copy si facand modificari atat in zona copiata cat si in zona de referinte. O parte din modificari vor ramane dupa incheierea executiei functiei, o parte nu.

```
def changeme4(vr):
  print(vr, id(vr))
  vr[1][0] = 4
  vr.append(8)
  print(vr, id(vr))


a = [30, [15]]
print(a, id(a))
changeme4(a.copy())
print(a, id(a))
```

```
[30, [15]] 132846027553984
[30, [15]] 132846025609344
[30, [4], 8] 132846025609344
[30, [4]] 132846027553984
```

In exemplul urmator se pierde complet referinta catre sublista [15,16,17] chiar daca atribuirea se face in interiorul functiei, chiar daca se foloseste copy pentru ca referinta interioara ramane aceeasi

```
def changeme5(vr):
  print(vr, id(vr))
  vr[2] = 4
  print(vr, id(vr))


a = [30, 32,[15,16,17],32]
print(a, id(a))
changeme5(a.copy())
print(a, id(a))
```

```
[30, 32, [15, 16, 17], 32] 132846026255808
[30, 32, [15, 16, 17], 32] 132846025297600
[30, 32, 4, 32] 132846025297600
[30, 32, [15, 16, 17], 32] 132846026255808
```

Pentru evitarea acestor probleme se poate folosi modulul deepcopy - acesta va duplica si referintele din lista astfel incat inlocuirea lor in interiorul functiei nu se va mentine la iesirea

```
import copy

def changeme6(vr):
  print(vr, id(vr))
  vr[1][0] = 4
  vr.append(8)
  print(vr, id(vr))


a = [30, [15]]
print(a, id(a))
changeme6(copy.deepcopy(a))
print(a, id(a))
```

```
[30, [15]] 132846025659712
[30, [15]] 132845688612416
[30, [4], 8] 132845688612416
[30, [15]] 132846025659712
```

```
def changeme7(vr):
  print(vr, id(vr))
  vr[2] = 4
  print(vr, id(vr))


a = [30, 32,[15,16,17],32]
print(a, id(a))
changeme7(a.copy())
print(a, id(a))
```

```
[30, 32, [15, 16, 17], 32] 132846594815104
[30, 32, [15, 16, 17], 32] 132846572861952
[30, 32, 4, 32] 132846572861952
[30, 32, [15, 16, 17], 32] 132846594815104
```

Capcana argumentelor default

In exemplul urmator, lista lst se creaza in realitate la definirea functiei si pastreaza aceeasi referinta pe toata durata executiei programului. Apelarea succesiva a functiei conduce la extinderea listei - comportament aproape sigur nedorit

```
def f(x, lst=[]):
    lst.append(x)
    return lst

print(f(1))
print(f(2))
print(f(3))
```

```
[1]
[1, 2]
[1, 2, 3]
```

Pentru a evita acest comportament, trebuie initializata lista cu o valoare predefinita "None"

```
def ff(x, lst=None):
    if lst is None:
        lst = []
    lst.append(x)
    return lst
print(f(1))
print(f(2))
print(f(3))
```

```
[1, 2, 3, 1]
[1, 2, 3, 1, 2]
[1, 2, 3, 1, 2, 3]
```

Accesul la variabilele definite in afara functiilor variabile globale

Python permite functiilor sa acceseze variabilele definite inainte de apelul oricarei functii. Aceasta inseamna ca orice variabila definita in afara functiei, inainte de apelarea acesteia va putea fi apelata in interiorul functiei dar nu va putea fi modificata

```
m = 32


def findme(a):
  print(m)

findme(12)
```

```
32
```

Daca variabila este declarata dupa apelul functiei, aceasta nu va fi disponibila ca referinta in interiorul acesteia.

```
def findme2(a):
  print(mmm)
```

```
findme2(12)
mmm = 32
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_1471/1992285719.py in <cell line: 0>()
      2   print(mmm)
      3
----> 4 findme2(12)
      5 mmm = 32

/tmp/ipykernel_1471/1992285719.py in findme2(a)
      1 def findme2(a):
----> 2   print(mmm)
      3
      4 findme2(12)
      5 mmm = 32

NameError: name 'mmm' is not defined
```

variabilele globale nu pot fi modificate in interiorul functiei ca si cum ar fi o variabila normala.

```
m = 32


def findme(a):
  print(m)
  m=21

findme(12)
```

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
/tmp/ipykernel_1471/2265294586.py in <cell line: 0>()
      6   m=21
      7
----> 8 findme(12)

/tmp/ipykernel_1471/2265294586.py in findme(a)
      3
      4 def findme(a):
----> 5   print(m)
      6   m=21
      7
```

```
UnboundLocalError: cannot access local variable 'm' where it is not
associated with a value
```

Pentru a modifica variabila globala in itneriorul functiei trebuie sa semnalam acest lucru. Aceasta se face, putin contraintuitiv, local (ca si in PHP de exemplu)

```
m = 32


def changemeingl(a):
  global m
  print(id(m))
  m = 21
  print(id(m))


print(id(m))
changemeingl(12)
print(id(m))
print(m)
```

```
11646344
11646344
11645992
11645992
21
```

Argumente pozitionale si argumente denumite (keyword)

Multe functii in Python permit trimiterea unor argumente specificand direct valoarea acesteia (sau o variabila care o detine) sau specificand o eticheta a argumentului, semnul egal si valoareea argumentului. De exemplu in urmatorul apel al functii print:

```
print(2, 1, 3, 4, sep=' <-> ')
```

```
2 <-> 1 <-> 3 <-> 4
```

primele argumente (2,1,3,4) se numesc argumente pozitionale (datorita faptului ca ordinea lor conteaza si tot ce le defineste in referintele in care vor fi reluate in itneriorul functiei este pozitia acestora) iar argumentul end=' ' este un argument de tip keyword

```
help(print)
```

```
Help on built-in function print in module builtins:

print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
```

```
      string inserted between values, default a space.
    end
      string appended after the last value, default a newline.
    file
      a file-like object (stream); defaults to the current sys.stdout.
    flush
      whether to forcibly flush the stream.
```

Daca privim help-ul functiei print observam ca sunt disponibile mnai multe argumente de tip keyword.

Pentru a defini parametrii de tip keyword intr-o functie trebuie sa fie numiti la momentul definirii acesteia

```
def mykwparfunc(one = 1, two = 0):
  print('one: ', one)
  print('two: ', two)
  print(' --- ')

mykwparfunc(one = 10, two = 12) #apel normal
mykwparfunc(two = 12, one = 10) # apel cu argumente inversate
mykwparfunc(50, 112) # apel cu argumente pozitionale
mykwparfunc(112, 50) # apel cu argumente pozitionale inversate
mykwparfunc(70, two = 212) # apel cu argumente mixte, respectand
pozitia
mykwparfunc()
```

```
one:  10
two:  12
 ---
one:  10
two:  12
 ---
one:  50
two:  112
 ---
one:  112
two:  50
 ---
one:  70
two:  212
 ---
one:  1
two:  0
 ---
```

daca incercam un apel cu argumente mixte dar nu respectam pozitia parametrului pozitional, vom capata o eroare:

```
mykwparfunc(70, one = 212) # apel cu argumente mixte, nerespectand
pozitia
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call
last)
/tmp/ipykernel_1471/1681946840.py in <cell line: 0>()
----> 1 mykwparfunc(70, one = 212) # apel cu argumente mixte,
nerespectand pozitia

TypeError: mykwparfunc() got multiple values for argument 'one'
```

Putem forta apelul cu argumente de tip keyword?

Python 3.8 a introdus o modificare relevanta in acest moment. Inainte de python 3.8 declaratia unei functii era astfel:

```
def functie(parametrii pozitionali sau keyword, *,
parametrii_care_sunt_exclusiv_keyword):
```

dupa python 3.8 declaratia astata astfel:

```
def name(parametrii_exclusiv_pozitionali, /,
parametrii_pozitionali_sau_kw, *, parametrii_exclusiv_kw):

def myallparfunc(pone,ptwo, /, one = 1, two = 0, *, three = 0,
four=0):
  print('pone: ', pone)
  print('ptwo: ', ptwo)

  print('one: ', one)
  print('two: ', two)
  print('three: ', three)
  print('four: ', four)
  print(' --- ')


myallparfunc(12,13)
```

```
pone:  12
ptwo:  13
one:  1
two:  0
three:  0
four:  0
 ---
```

```
myallparfunc(12,13,14,15)
```

```
pone:  12
ptwo:  13
one:  14
two:  15
three:  0
four:  0
 ---

myallparfunc(12,13,14,15,16,17)

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_1471/4148942032.py in <cell line: 0>()
----> 1 myallparfunc(12,13,14,15,16,17)

TypeError: myallparfunc() takes from 2 to 4 positional arguments but 6 were given
```

Observam ca:

parametrii dinainte de / sunt exclusiv pozitionali, acestia sunt obligatorii

- Parametrii din mijloc (dintre / si *) sunt semi-pozitionali care au valori predefinite, functia poate fi apelata cu acesti parametrii ca parametrii pozitionali sau ca parametrii de tip keyword
- Parametrii finali (dupa *) sunt parametrii de tip keyword care nu pot fi apelati decat ca parametrii de tip keywords.
- Ultimii doi parametrii (three, four) trebuie obligatoriu sa fie chemati cu numele lor.

La definirea functiei trebuie tinut cont de tipurile de date necesare si de modul de apelare dorit. Parametrii de tip exclusiv keyword pot fi utilizati pentru a specifica o lista de valori predefinite a caror modificare poate fi dorita sporadic. Aceasta ii face sa poata fi evitati de la majoritatea apelarilor, fara consecinte asupra executiei functiei datorita existentei valorilor predefinite.

Functii cu numar variabil de argumente

Sunt situatii in care nu stim exact cate argumente va primi functia. Putem defini functii care vor primi un numar variabil de argumente

Functii cu numar variabil de parametrii pozitionali

```
def suma(*numere):
    suma = 0
    for num in numere:
        suma = suma + num
    return suma

print(suma(12,34,56))
```

```
print(suma(12))
print(suma(12,34,56,54,67,897,23,54,23))
```

```
102
12
1220
```

Ar fi interesant de vazut exact ce tip are variabila "numbers". Daca ar fi sa prezicem, ce tip nativ Python permite alaturarea ordonata, cu posibilele dubluri a mai multor elemente, posibil de tipuri diferite? Acest tip este tuplul. Putem verifica, rescriind functia astfel incat sa imprime tipul variabilei "numere"

```
def suma_t(*numere):
    print (type(numere))
    suma = 0
    for num in numere:
        suma = suma + num
    return suma

print(suma_t(12,34,56))
print(suma_t(12))
print(suma_t(12,34,56,54,67,897,23,54,23))
```

```
<class 'tuple'>
102
<class 'tuple'>
12
<class 'tuple'>
1220
```

Argumentele trimise catre functie sunt puse intr-un tuplu iar acesta este "despachetat" in interiorul functiei. Despachetarea permite variabile cu stea, variabile care vor acumula tot ce ramane neatribuit in cele dinaintea lor

Functii cu numar variabil de parametrii de tip keyword

O functie poate astepta un numar variabil de parametrii de tip keyword.

```
def calculate_total_cost(price, **kwargs):
    tax_rate = kwargs.get('tax_rate', 0.05)
    discount = kwargs.get('discount', 0)
    shipping_cost = kwargs.get('shipping_cost', 5.0)

    total_cost = price + (price * tax_rate) - discount + shipping_cost
    return total_cost

# Example usage
price = 100.0
total_cost = calculate_total_cost(price, tax_rate=0.08, discount=10.0,
```

```
shipping_cost=8.0)
print(f"Total cost: {total_cost}")
```

```
Total cost: 106.0
```

Evident, ca si in cazul parametrilor pozitionali variabili, o lista variabila de parametrii, se poate inlocui cu un singur parametru de tip dictionar